

OPENSSL CERTIFICATE SIGNING REQUEST LAB

Linux • Public Key Cryptography • Certificate Enrollment Preparation

Environment	Ubuntu Linux VM running in Oracle VirtualBox
Tool	OpenSSL command-line utility
Subject hostname shown	ubuntu.cybergregory.lab
Public key	RSA, 2048-bit
Evidence status	CSR details screenshot supplied; separate signature and key-match validation should be captured to complete the evidence set.

Lab objective

Generate a private key and Certificate Signing Request (CSR) with OpenSSL, inspect the request fields, and validate the request before submitting it to a trusted Certificate Authority (CA). This exercise demonstrates certificate-enrollment preparation; it does not itself issue a certificate.

Skills demonstrated

- Linux terminal operation and file handling
- OpenSSL CSR generation and inspection
- RSA public-key concepts
- CSR signature validation and evidence collection
- Security-minded handling of private keys

1. Procedure

Commands below document the workflow. Run and capture the verification commands in the VM if you have not already done so.

Generate the private key and CSR

```
openssl req -new -newkey rsa:2048 -nodes \\n -keyout server.key \\n -out server.csr
```

Inspect the CSR

```
openssl req -in server.csr -noout -text
```

Verify the CSR signature

```
openssl req -in server.csr -noout -verify
```

Compare public-key hashes

```
openssl pkey -in server.key -pubout -outform DER | sha256sum\nopenssl req -in server.csr -pubkey -noout | openssl pkey -pubout -outform DER | sha256sum
```

Expected signature verification output

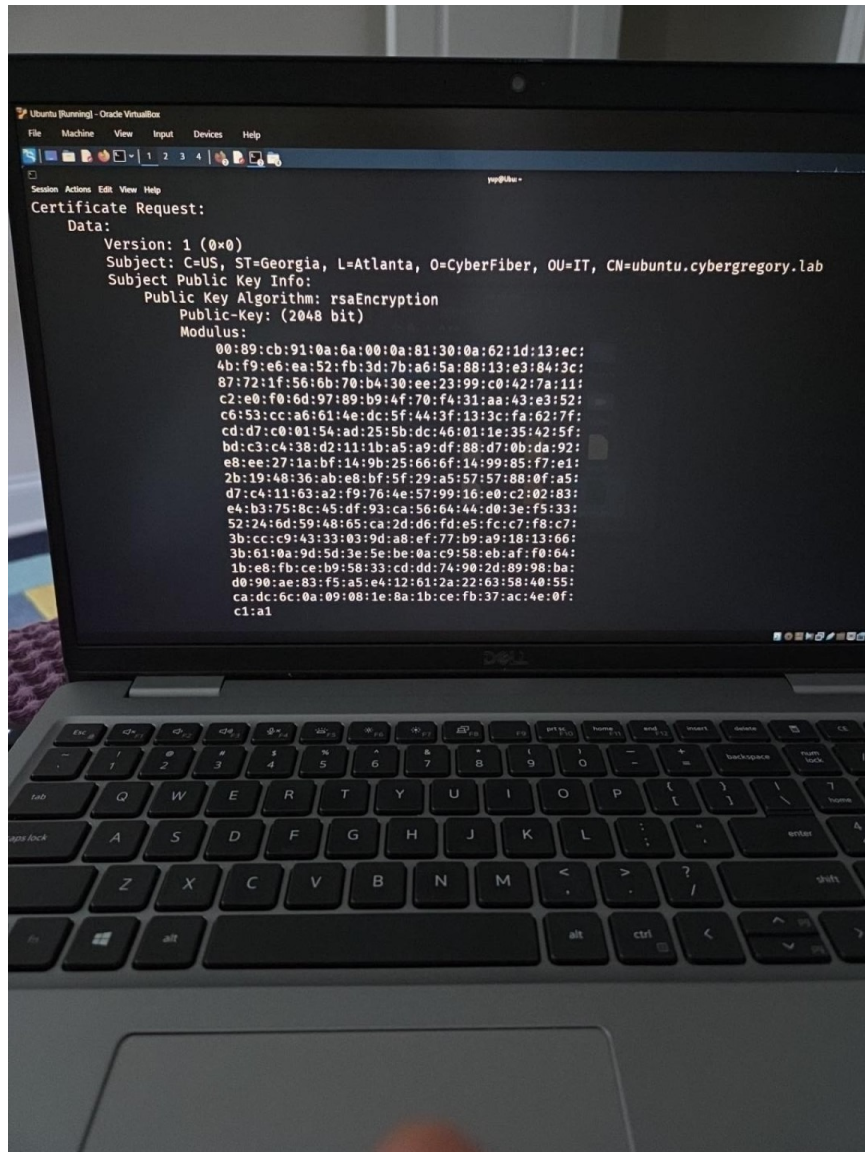
```
Certificate request self-signature verify OK
```

The two public-key hashes should be identical. Record a pass only after running the commands and confirming the result.

2. Observed evidence

The supplied screenshot shows OpenSSL displaying a CSR with Version 1 request data, subject fields, and a 2048-bit RSA public key. The visible subject includes C=US, ST=Georgia, L=Atlanta, O=CyberFiber, OU=IT, and CN=ubuntu.cybergregory.lab.

Evidence A — CSR details displayed in Ubuntu terminal



The supplied image supports CSR inspection. It does not by itself prove the separate signature-verification command succeeded.

3. Results and interpretation

Confirmed from screenshot: OpenSSL displayed the CSR subject and 2048-bit RSA public-key information.

Still to document: capture the signature verification command and successful output, then compare the two public-key hashes. These results are not inferred from the screenshot.

Certificate status: no CA-issued certificate is evidenced here. A CSR is a request, not a signed server certificate.

4. Security considerations

- Keep server . key confidential; never commit it to GitHub or include it in a portfolio package.
- The -nodes option creates an unencrypted private key. Restrict file permissions and consider encrypted key storage where appropriate.
- Production TLS certificates should include suitable Subject Alternative Name (SAN) entries.
- Use a trusted CA and follow the applicable certificate policy before deployment.

5. Portfolio completion checklist

- Capture CSR details with `openssl req -in server.csr -noout -text`.
- Capture signature verification with `openssl req -in server.csr -noout -verify`.
- Compare public-key hashes and record the observed result.
- Keep private keys and credentials out of the repository.
- Add any generated text report if available.

Conclusion

This lab documents the initial CSR-generation and inspection workflow on Ubuntu Linux. The supplied screenshot demonstrates the subject information and 2048-bit RSA public key. Add the separate verification evidence before claiming that the full validation workflow is complete.

Before publishing, confirm that the example organization and hostname are appropriate to share. Do not publish private keys, credentials, or sensitive internal hostnames.